Sprint 0 Report

System Architecture

The system follows a **Client-Server architecture**, with the browser-based Next.js / React frontend acting as the client and all backend services operating server-side. The backend itself is organized as a **layered architecture,** incoming requests from the client pass through the *Auth Layer* through the Supabase Auth first, which validates access before anything else happens. Authenticated requests then reach the *Business Logic* layer, which handles all core operations such as video management, watch history, and custom lists. The Business Logic layer communicates downward to the *Data Layer* at Supabase DB for all persistent storage and session management. This strict top-down dependency flow, client → auth → logic → data, is what defines the layered pattern and keeps each layer's responsibility clearly bounded.

External services integrate at the layer most relevant to their concern. Google OAuth connects at the Auth layer, handling third-party authentication. The Email Service connects at the Business Logic layer, triggered by application events such as account verification and password reset.

The AI Module operates as a self-contained component outside the core layers. At ingestion time, video content is processed and embedded into a searchable representation. At query time, the user's natural language input is matched against those embeddings to return ranked, timestamped results. The module communicates with the Core Platform through a straightforward request-response pattern.

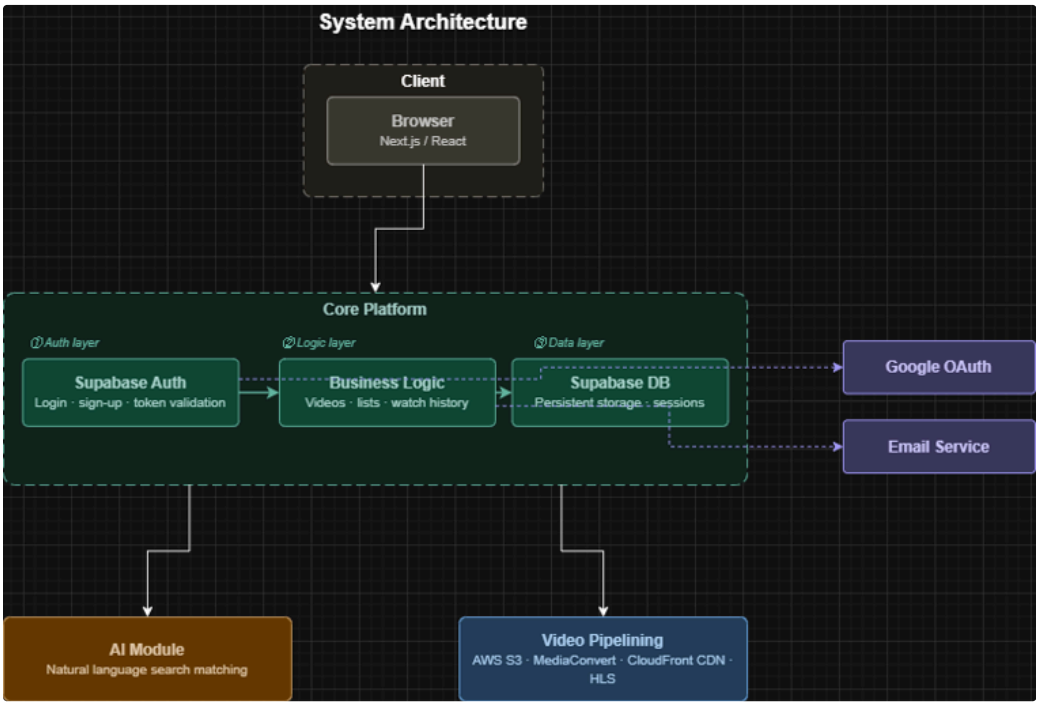


Figure 1: System Architecture

The video ingestion and delivery subsystem follows an event-driven pipeline pattern, distinct from the layered core. A video upload to AWS S3 triggers an automated transcoding job via AWS MediaConvert, which produces multiple resolution outputs (360p, 720p, 1080p) packaged as HLS segments with a manifest file. These processed files are stored back in S3 and served globally through CloudFront CDN. The HLS player in the browser then implements adaptive bitrate streaming (ABR), dynamically selecting the appropriate resolution based on the viewer's available bandwidth, ensuring smooth playback without manual configuration.

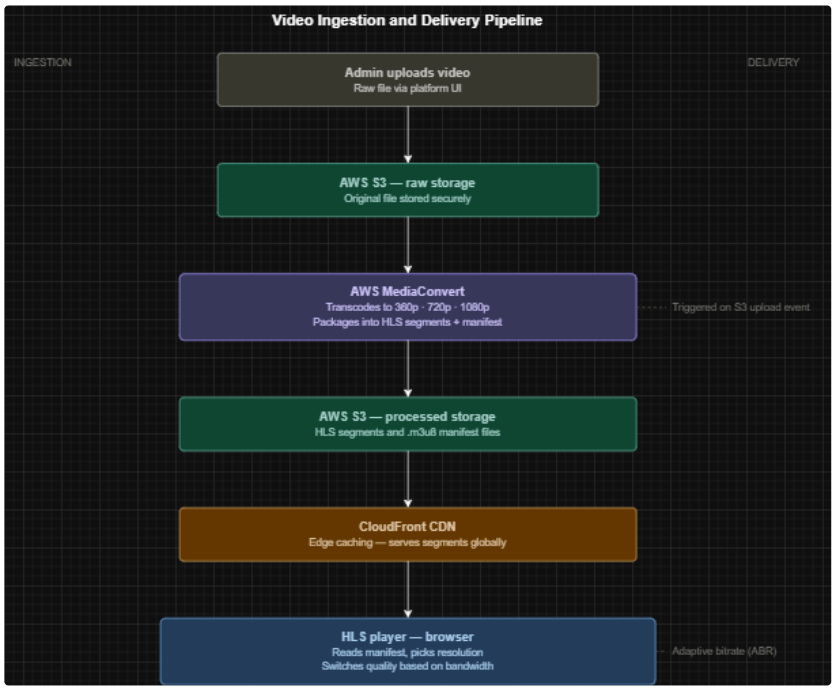


Figure 2: Video Ingestion and Delivery Pipeline

Together, the following patterns define the overall system architecture:

- **Client-Server:** Browser (Next.js / React) as client; all backend services server-side.
- **Layered Backend:** Auth → Business Logic → Data, with strict top-down dependency flow.
- **Event-Driven Pipeline:** S3 upload event triggers MediaConvert transcoding asynchronously.
- **Self-Contained AI Module:** Natural language search via request-response, independent of core layers.

Refined Product Backlog & Sprint 1 Backlog

SoftwareTeam | Backlog

Something went wrong on our end

If this keeps happening, share this information with your admin, who should contact support.

```
Hash 1YPF8E1
Trace a53f41b11238cf0ccd8346f5b42194d7
```

Reload page

Documentation of Planning Poker Session

Shirts estimate

User Story	Amer	Anan	Moaz	Ali - Hassan	Ali - Moustafa	Seba	Result	Rounds
5	3	3	5	5	13	5	3	2
6	8	5	5	8	8	8	8	1
7	40	20	20	20	20	20	20	1
8	13	8	5	5	5	8	5	1
9	20	20	40	20	13	20	20	2
10	13	5	5	3	8	8	5	1
11	13	5	5	3	8	5	5	1
15	5	5	5	5	5	5	5	1
16	3	5	5	3	3	3	3	1
17	8	40	8	3	8	5	8	2
18	8	8	13	13	8	8	13	1
19	8	8	8	8	8	8	8	1
20	5	3	8	3	5	5	8	1
21	1	2	2	2	3	2	2	1
22	8	13	13	5	5	5	8	1
23	5	20	8	5	13	8	13	1
24	5	13	5	8	5	8	8	1
25	5	8	5	5	5	5	5	1
26	5	5	8	5	5	8	5	1
30	5	8	5	13	8	5	5	2
31	8	5	8	13	13	8	8	2

Sheet1

Chosen Stories in Sprint 1 Backlog [23 Points]

We select stories that work towards creating the MVP. Selection prioritized stories that deliver core user-facing value, minimize blockers for dependent features, and introduce the team to key technical domains early. Stories not included in this sprint were deliberately deferred to later sprints as they build upon the MVP baseline being established here.

- Story 1: [SCRUM-5] As a user (viewer), I want to sign up for an account on the platform, so that I can have a personalized viewing experience. **[3 Points]**

  This story was selected because signing up for an account is the first think a user would think about when using a platform to have a personalized experience. Also, it is a very basic feature and not complex.

- Story 2: [SCRUM-30] As an admin, I want to upload a video file with its metadata through the dashboard, So that the movie is recorded in the system and queued for processing. **[5 Points]**

  Although it has relatively high complexity, this story was selected because all other functionalities of the platform depend on the existence of movies on the platform which need to be uploaded by the admin. From a risk perspective, this story introduces the highest technical risk in the sprint — file handling, storage integration, and metadata management are areas prone to unexpected complexity. Tackling it in Sprint 1 allows us to surface and resolve these risks early, before dependent features are built on top of it.

- Story 3: [SCRUM-11] As a viewer on the streaming platform, I want videos to be organized into clear genre-based rows, so that I can easily browse a specific theme and discover more videos without leaving the main page. **[5 Points]**

  This story was selected because it's essential for the streaming platform to organize the videos in a way that is relevant to the user.

- Story 4: [SCRUM-15] As a user (viewer), I want to create and switch between profiles, so that each person in my household gets their own recommendations and watch history. **[5 Points]**

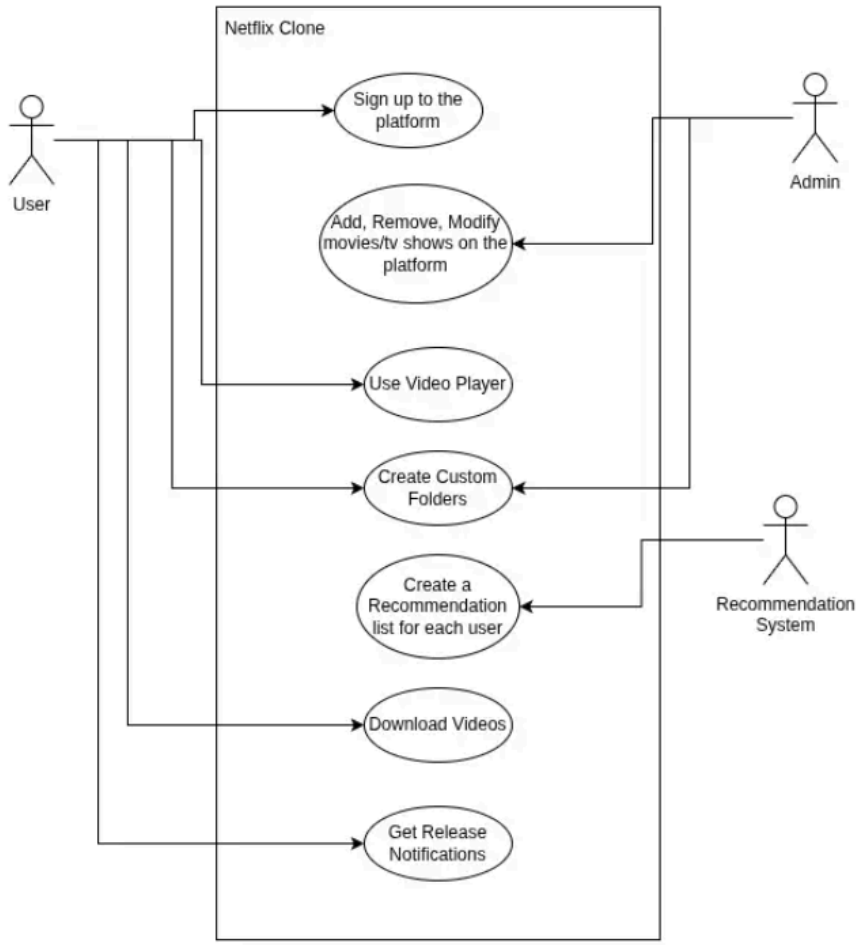
  This story was selected because it enables ease of use for the user such that each person can follow

their respective content.

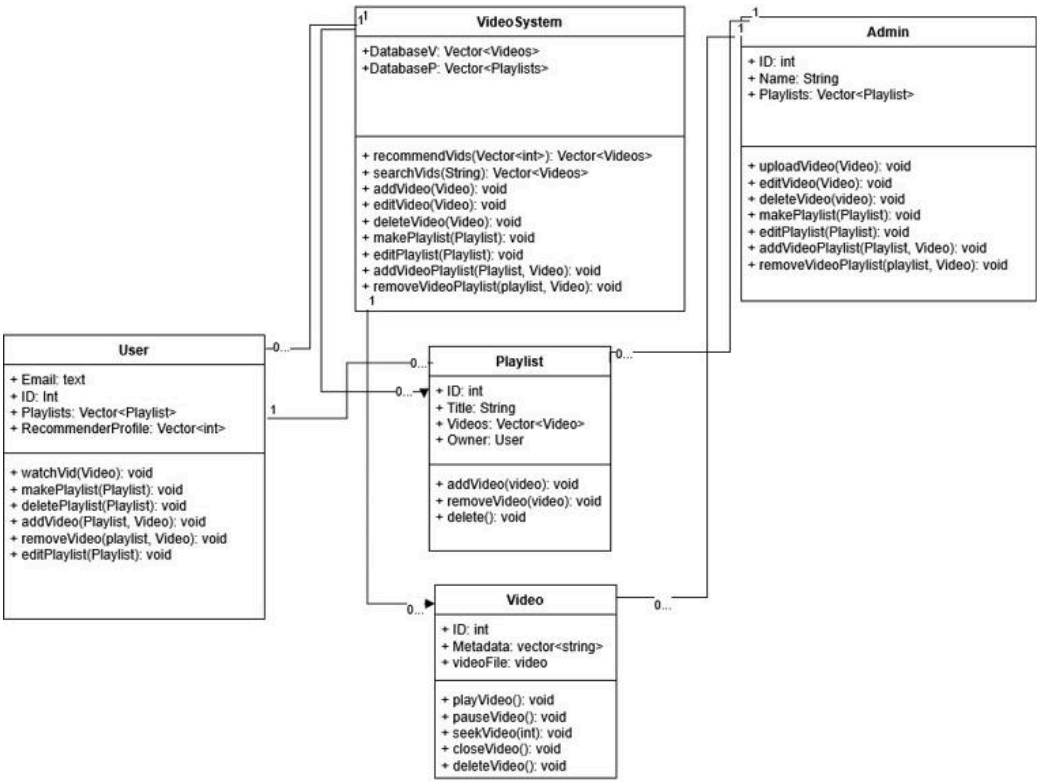
- Story 5: [SCRUM-26] As a viewer, I want to play, pause, and jump to any point on the video timeline, so that I can control when I watch and freely manage my viewing session without having to watch content I've already seen. **[5 Points]**

  This story was selected because it represents the most important functionality to have in a streaming platform which is basic playback and video control functionality, and other features depend on its existence. This is a valuable learning experience for us because it introduces us to streaming protocol and and player libraries.

Use Case Diagram

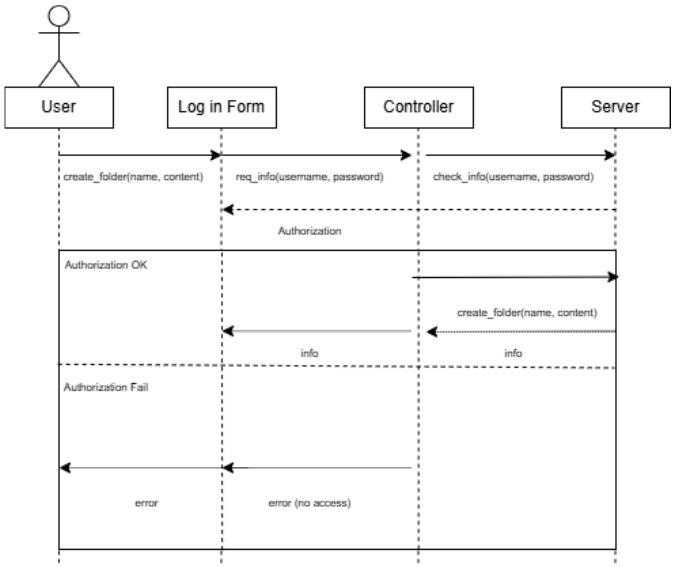


Class Diagram

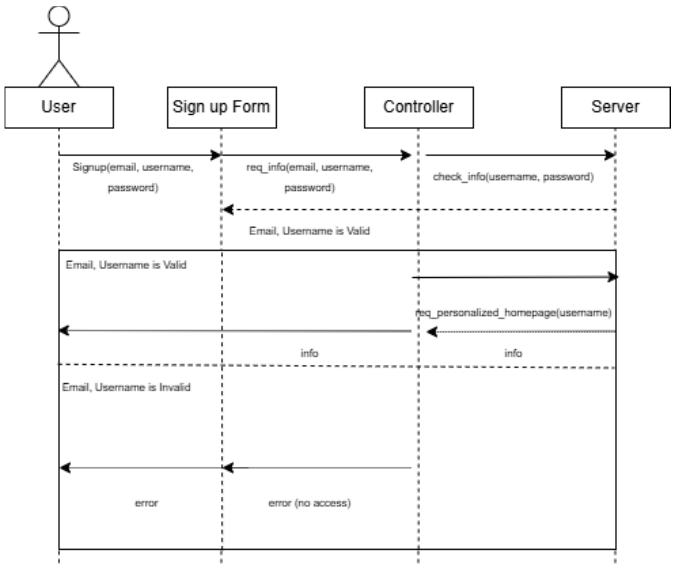


Class Diagram

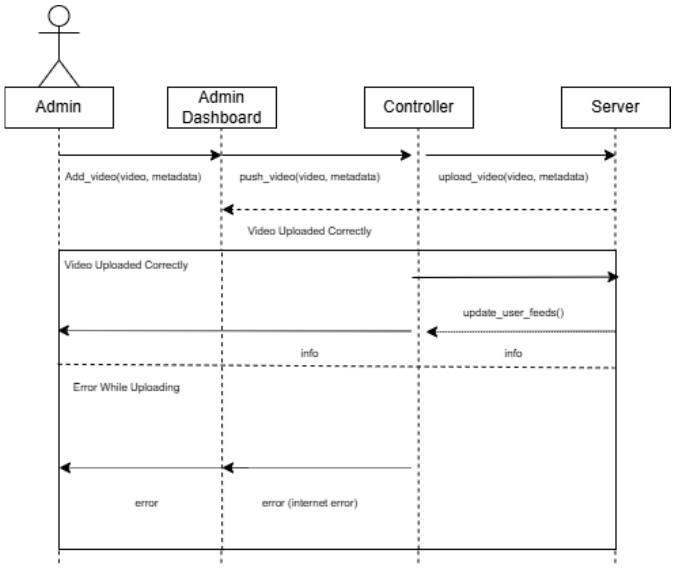
Sequence diagrams for the top 3 features



Feature 1: Log in



Feature 2: Sign Up



Feature 3: Add Video

User Manual

User Manual for Loopy

Share

User Manual for Loopy

By Seba Wahba 3 min 5

🌀 The Loopy Explorer's Guide: Your Technical Manual & Adventure Map

🚀 Level 1: Your Gateway to the Loop (Authentication)

Welcome to Loopy! This isn't just a user manual; it's your field guide to mastering our AI-powered video streaming platform. Before we dive into the advanced AI features, we need to get you securely logged in.

🔑 Choose Your Entrance Protocol

Loopy offers two secure methods to access your personalized dashboard:

•The Fast Track (Google OAuth): Click "Continue with Google" for a flawless, one-click authentication process. This is highly recommended for speed and security.

•The Classic (Standard Account): Prefer to keep things separate? Create a dedicated Loopy ID using your email address and a robust, strong password.

Interactive Challenge: Try logging in with Google; it utilizes secure OAuth 2.0 in collaboration

Link to Figma prototype

Video Streaming App Design

Links:

Confluence Report (if anything is not working, please open it)

SDS

SRS

Jira

User Manual

Prototype